Performance Analysis of Offset-based L2 Prefetching and Exclusive Cache Implementation

Project Assignment (PA-2)

Team AVM

Manish Kumar

Aditya Gaikwad

Vedant Chavan

October 2025

Abstract

This report presents the implementation details and experimental results for three tasks performed as part of PA-2 for CS683: (1) an offset-based L2 data prefetcher (non-inclusive cache), (2) an exclusive-cache implementation and its performance comparison with a non-inclusive cache, and (3) the adaptation and evaluation of the offset-based prefetcher for the exclusive cache hierarchy. Experiments are performed using the ChampSim simulator across four supplied traces.

Contents

1 Introduction **3**

2 Experimental Setup **3**

3 Task 1: Offset-based L2 Prefetcher (Non-Inclusive Cache) **3**

3.1 Implementation Overview 3

3.2 Results (Non-Inclusive) 3

4 Task 2: Exclusive Cache Implementation **5**

4.1 Implementation Overview 5

4.2 Results: Non-Inclusive vs Exclusive (No Prefetcher) 5

5 Task 3: Offset-based L2 Prefetcher for Exclusive Cache **6**

5.1 Implementation Overview 6

5.2 Results (Exclusive) 7

6 Discussion **9**

1 Introduction

Prefetching and cache policy design are critical for improving memory subsystem performance. In this assignment we implemented an offset-based L2 prefetcher, implemented an exclusive cache hierarchy, and adapted the prefetcher to work correctly under exclusivity constraints. We evaluated all changes using ChampSim and report metrics such as IPC and MPKI for L1D, L2 and LLC, along with speedups relative to a baseline (no prefetcher) configuration.

2 Experimental Setup

- **Simulator:** ChampSim (course-provided configuration)
- **Traces:** trace1.champsimtrace.xz, trace2.champsimtrace.xz, trace3.champsimtrace.xz, trace4.champsimtrace.xz
- **Metrics:** IPC, L1D MPKI, L2 MPKI, LLC MPKI, Simulation time (seconds)
- **Prefetchers compared:** Baseline (no prefetcher), IP-Stride, Offset-based (32, 64, 128 entries in region/offset table)
- **Cache policies:** Non-inclusive baseline, Exclusive cache implementation

3 Task 1: Offset-based L2 Prefetcher (Non-Inclusive Cache)

3.1 Implementation Overview

The prefetcher maintains a `RegionEntry` table keyed by a page signature. Each entry stores up to five offsets and associated statistics: *frequency*, *prefetches issued*, and *useful prefetches*. A score is computed for each offset as:

$$\text{score} = \text{offset_frequency} \times \text{offset_accuracy},$$

where $\text{offset_accuracy} = \frac{\text{useful prefetches}}{\text{total prefetches}}$. Offsets with frequency ≥ 3 are eligible for prefetching. Offsets are sorted by score (ties broken by frequency), and a degree of prefetching is chosen based on rank.

To measure prefetch usefulness, the simulator was extended to indicate, on a cache hit, whether the block was previously prefetched. This value is used to update the useful-prefetch counters and compute accuracy.

3.2 Results (Non-Inclusive)

The following tables summarize the results gathered for Task 1. For each trace we provide L2 MPKI, IPC and the speedup relative to the baseline (no prefetcher). Figures are provided as placeholders for the speedup plots.

Trace 1

Table 1: Performance for `trace1.champsimtrace.xz` (Non-Inclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	40.4106	0.528 577	1.000
IP-Stride	40.3407	0.546 480	1.034
Offset (32 entries)	40.3904	0.601 349	1.138
Offset (64 entries)	40.3904	0.601 349	1.138
Offset (128 entries)	40.3986	0.601 178	1.137

Trace 2

Table 2: Performance for `trace2.champsimtrace.xz` (Non-Inclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	47.6970	0.463 132	1.000
IP-Stride	47.7267	0.478 241	1.033
Offset (32 entries)	47.7168	0.516 006	1.114
Offset (64 entries)	47.7168	0.516 006	1.114
Offset (128 entries)	47.7168	0.516 006	1.114

Trace 3

Table 3: Performance for `trace3.champsimtrace.xz` (Non-Inclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	22.1023	0.838 345	1.000
IP-Stride	22.1965	0.852 268	1.016
Offset (32 entries)	22.1022	1.032 880	1.232
Offset (64 entries)	22.1022	1.032 880	1.232
Offset (128 entries)	22.1022	1.032 880	1.232

Trace 4

Table 4: Performance for `trace4.champsimtrace.xz` (Non-Inclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	47.5620	0.461 042	1.000
IP-Stride	47.5704	0.479 230	1.039
Offset (32 entries)	47.5802	0.516 168	1.119
Offset (64 entries)	47.5802	0.516 168	1.119
Offset (128 entries)	47.5848	0.515 731	1.119

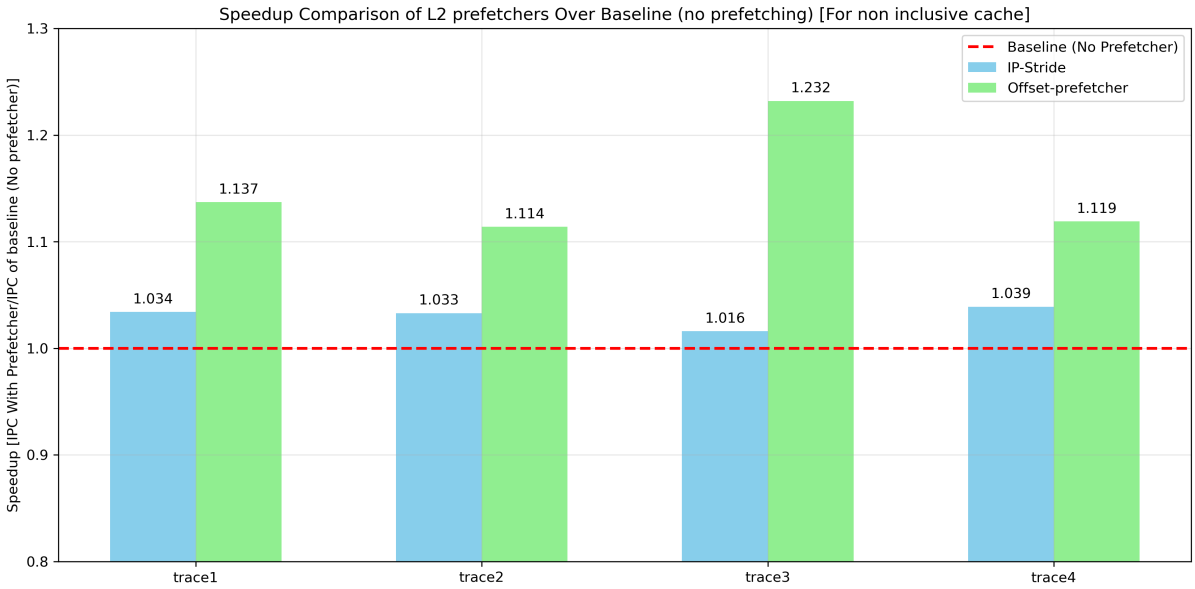

Figure 1: Speed Up Plot of IP-Stride L2 prefetcher and Offset based L2 Prefetcher (With non inclusive cache).

4 Task 2: Exclusive Cache Implementation

4.1 Implementation Overview

To enforce exclusivity across levels the cache control flow was modified at multiple points:

1. **handle_fill:** Bypass fills when the block's intended fill level is lower than the current level; avoid evictions and writeback queue insertions when bypassing. When a block is filled at an upper level and the block exists in a lower level (LLC), the block is invalidated in the lower level to preserve exclusivity.

2. **handle_writeback:** Introduced a `valid` bit in the `PACKET` structure to allow invalidation of writeback queue entries. Skip writeback when an entry is invalid.

3. **add_rq / add_wq:** When a read finds the requested block in a writeback queue, the block is returned to the requester and the writeback entry invalidated. Merging of subsequent writes to an invalid entry re-validates that entry rather than creating duplicates.

Asserts were placed at strategic points (handle_fill, handle_read, writeback) to ensure correctness of exclusivity invariants.

4.2 Results: Non-Inclusive vs Exclusive (No Prefetcher)

The table below compares baseline (no prefetcher) metrics for the non-inclusive and exclusive cache configurations across the four traces.

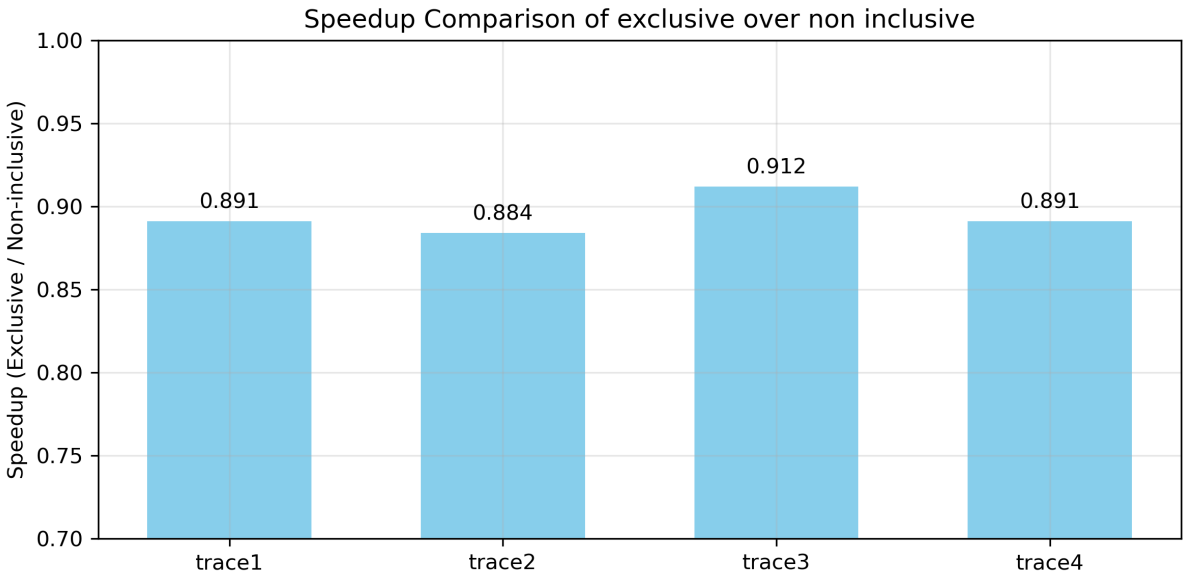

Figure 2: Speedup of Exclusive cache relative to Non-Inclusive (baseline).

Table 5: Baseline (No prefetcher): Non-Inclusive vs Exclusive

Config	Trace	IPC	L1D MPKI	L2 MPKI	LLC MPKI
Non-Inclusive	1	0.528577	66.0504	40.4106	40.1095
Exclusive	1	0.470898	66.0504	66.0504	40.2379
Non-Inclusive	2	0.463132	78.0221	47.6970	47.3797
Exclusive	2	0.409313	78.0226	78.0219	47.5020
Non-Inclusive	3	0.838345	35.7547	22.1023	21.7124
Exclusive	3	0.764891	35.7547	35.7547	21.9610
Non-Inclusive	4	0.461042	77.7241	47.5016	47.1967
Exclusive	4	0.410739	77.7234	77.7233	47.2719

5 Task 3: Offset-based L2 Prefetcher for Exclusive Cache

5.1 Implementation Overview

The offset prefetcher was adapted for exclusivity with the following changes:

- The same `RegionEntry` design and scoring mechanism are used (score = frequency × accuracy). Only offsets with frequency ≥ 3 are eligible.
- Prefetches are issued only after verifying that the target block is not present in the current cache level, any lower levels, existing MSHRs, or writeback queues. This preserves exclusivity and avoids redundant prefetches.
- A negative-feedback mechanism was introduced: writeback events decrement the frequency of offsets observed during writebacks (to reduce issuing prefetches for data that is being evicted frequently).
- The simulator's prefetch accounting is updated to pass a flag indicating whether a cache fill originated from a writeback so that the prefetcher can update statistics appropriately.

5.2 Results (Exclusive)

The results below summarize the performance of the offset prefetcher under the exclusive cache configuration.

Trace 1

Table 6: Performance for `trace1.champsimtrace.xz` (Exclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	66.0504	0.470 898	1.000
IP-Stride	69.5935	0.476 070	1.011
Offset (32 entries)	72.5475	0.518 200	1.101
Offset (64 entries)	72.5475	0.518 200	1.101
Offset (128 entries)	72.5529	0.517 978	1.099

Trace 2

Table 7: Performance for `trace2.champsimtrace.xz` (Exclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	78.0219	0.409 313	1.000
IP-Stride	81.8336	0.415 526	1.014
Offset (32 entries)	85.5389	0.442 335	1.080
Offset (64 entries)	85.5389	0.442 335	1.080
Offset (128 entries)	85.5389	0.442 335	1.080

Trace 3

Table 8: Performance for `trace3.champsimtrace.xz` (Exclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	35.7547	0.764 891	1.000
IP-Stride	38.2053	0.763 002	0.996
Offset (32 entries)	39.6532	0.915 658	1.197
Offset (64 entries)	39.6532	0.915 658	1.197
Offset (128 entries)	39.6532	0.915 658	1.197

Trace 4

Table 9: Performance for `trace4.champsimtrace.xz` (Exclusive)

Prefetcher Type	L2 MPKI	IPC	Speedup
No (baseline)	77.7233	0.410 739	1.000
IP-Stride	81.5279	0.417 098	1.015
Offset (32 entries)	85.2359	0.444 140	1.081
Offset (64 entries)	85.2359	0.444 140	1.081
Offset (128 entries)	85.2158	0.444 088	1.081

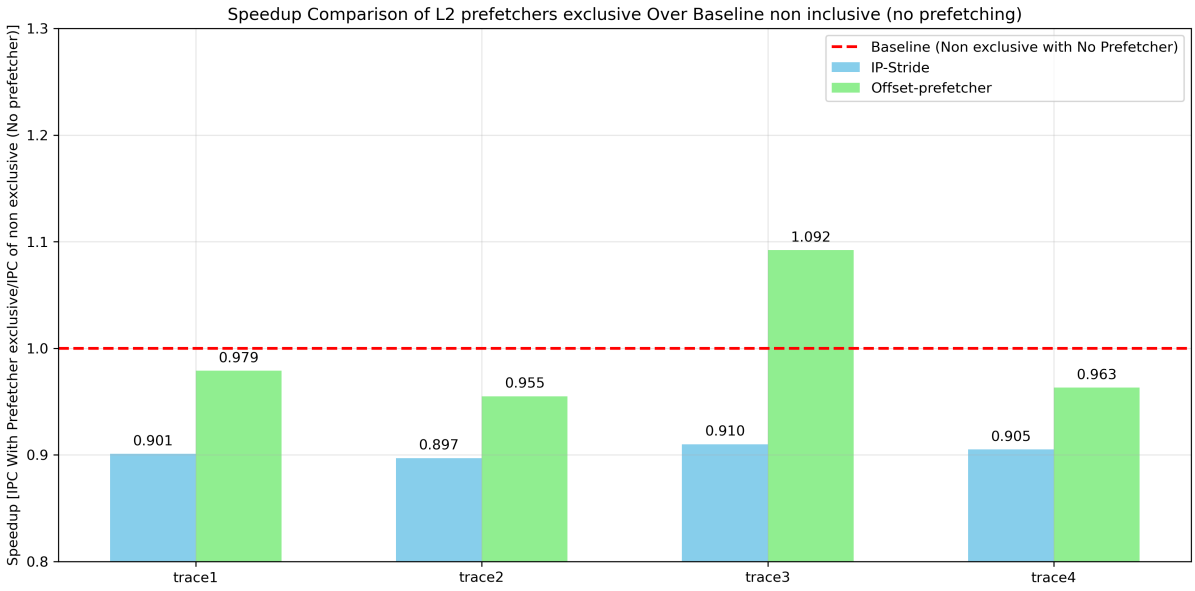

Figure 3: Speed Up Plot of IP-Stride L2 prefetcher and Offset based L2 Prefetcher (With non-inclusive cache baseline

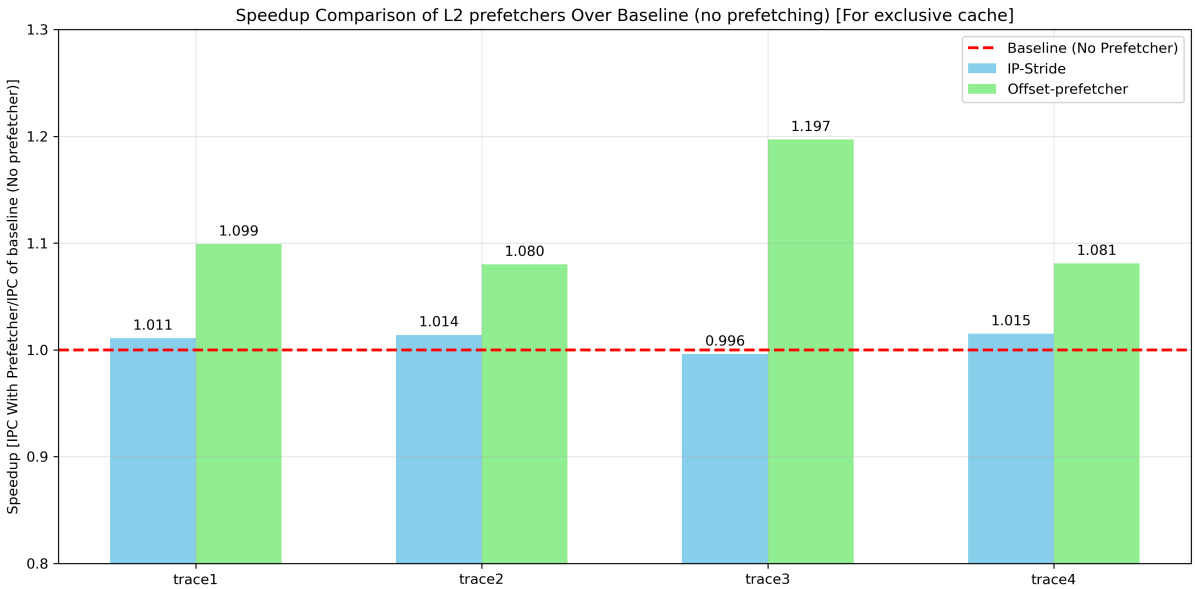

Figure 4: Speed Up Plot of IP-Stride L2 prefetcher and Offset based L2 Prefetcher (With exclusive cache baseline

6 Discussion

Across the experiments the offset-based prefetcher often improves IPC notably over the baseline and IP-stride prefetcher, particularly for traces where the workload benefits from predictable region/offset behavior. In the exclusive cache configuration, overall L2 MPKI numbers increase (as expected) due to exclusivity moving data out of lower levels; however, the offset prefetcher still achieves measurable IPC improvements by prefetching useful data and by using negative feedback on writebacks to avoid harmful prefetches.

Acknowledgements

We thank the course instructors and teaching assistants for providing the ChampSim framework and guidance for PA-2.

Appendix: Notes on Reproducibility

All experiments were executed on the ChampSim framework with the configuration provided for the assignment. Metric extraction and table generation were done from the ChampSim output metric files. The accompanying spreadsheet used during analysis is available at: https://docs.google.com/spreadsheets/d/13Ah9X9RHfPFDFB2k1NE1cPHaNnJIb9G8SCVfv659ur0/edit?usp=sharing. Github repository will be made public here: https://github.com/flameberry/CS683PA2.